DOKUMENTASI TEKNIS

PLANNING & ARCHITECTURE DESIGN

Multi-Tier Edge Caching System
Portal Berita Nasional (Breaking News)

Pertemuan 1
Kelompok 5
Repositori GitHub:
github.com/febrisariamalia/breaking-news-caching

Informasi Dokumen

Elemen	Keterangan
Nama proyek	Multi-Tier Edge Caching System: Portal Berita Nasional (Breaking News)
Fokus pertemuan	Planning & Architecture Design
Luaran	Diagram arsitektur, skema jaringan Docker, spesifikasi awal Docker Compose, dan dokumentasi repositori Git
Repositori	https://github.com/febrisariamalia/breaking-news-caching.git

Dokumen ini membahas perencanaan dan rancangan arsitektur berdasarkan struktur serta konfigurasi yang dijelaskan pada repositori proyek. Rancangan dan target kinerja tidak dianggap sebagai hasil pengujian sebelum divalidasi melalui implementasi.

1. Deskripsi Proyek

Proyek Multi-Tier Edge Caching System dirancang sebagai infrastruktur portal berita yang dapat menghadapi lonjakan permintaan saat peristiwa tertentu menjadi perhatian publik. Desain menggunakan beberapa lapisan cache untuk mengurangi permintaan yang mencapai aplikasi asal dan menekan pengambilan data berulang.

Komponen utama terdiri atas Varnish Cache sebagai edge HTTP cache, Nginx sebagai reverse proxy, aplikasi web sebagai pemroses permintaan, serta Redis sebagai application data cache. Layanan didefinisikan menggunakan Docker Compose dan dihubungkan melalui jaringan bridge Docker bernama news-network.

2. Planning Sistem

2.1 Permasalahan dan alasan perancangan

Lonjakan trafik dapat meningkatkan beban CPU, memori, dan koneksi pada aplikasi asal. Jika setiap permintaan diproses dari awal, waktu respons berpotensi meningkat. Karena itu, sistem dirancang untuk melayani permintaan berulang dari cache dan meneruskan hanya permintaan yang memerlukan pemrosesan lebih lanjut.

2.2 Tujuan perancangan

- Mengurangi permintaan yang mencapai origin melalui Varnish Cache.
- Mengurangi pengambilan data berulang melalui Redis.
- Mengatur penerusan permintaan menggunakan Nginx.
- Memisahkan komponen melalui container Docker.
- Mengatur dependensi, health check, dan batas sumber daya.
- Menyiapkan fondasi pengujian beban dengan target skenario hingga 1.000 Virtual Users (VU).

Target 1.000 VU merupakan sasaran pengujian, bukan bukti bahwa sistem telah mencapai kapasitas tersebut.

2.3 Komponen teknologi

Komponen	Teknologi / konfigurasi	Fungsi
Edge cache	Varnish 7.3	Menyimpan dan melayani respons HTTP dari cache.
Reverse proxy	Nginx Alpine	Meneruskan permintaan ke aplikasi.
Aplikasi	Build dari ./app	Menjalankan logika aplikasi dan berkomunikasi dengan Redis.
Application cache	Redis 7 Alpine	Menyimpan data cache untuk digunakan kembali.
Orkestrasi	Docker Compose	Mendefinisikan layanan, dependensi, jaringan, dan sumber daya.

Version control	Git/GitHub	Mengelola kode, konfigurasi, dokumentasi, dan riwayat perubahan.

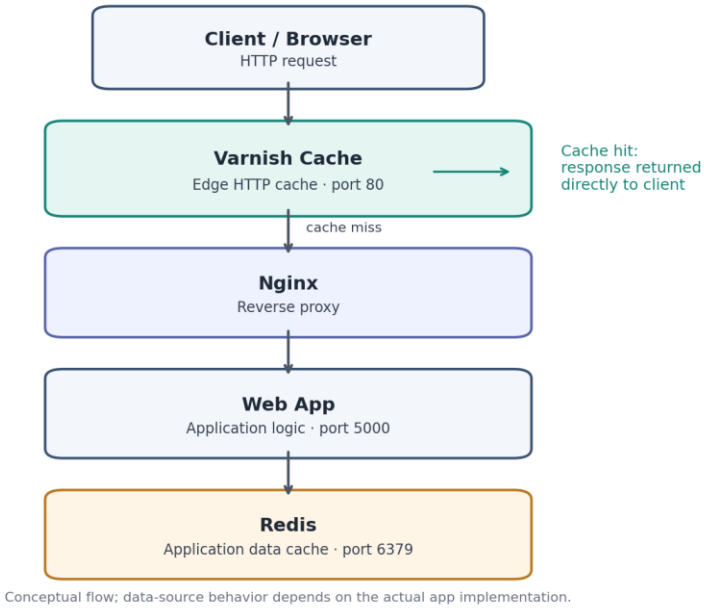
2.4 Alur permintaan

1. Pengguna mengirimkan permintaan HTTP melalui port 80 pada host.
2. Varnish memeriksa apakah respons tersedia dan masih valid dalam cache.
3. Pada cache hit, Varnish mengembalikan respons langsung kepada pengguna.
4. Pada cache miss, permintaan diteruskan ke Nginx.
5. Nginx meneruskan permintaan ke aplikasi web pada port internal 5000.
6. Aplikasi menggunakan Redis melalui hostname redis dan port 6379 untuk mendukung caching data.
7. Aplikasi memproses permintaan dan respons dikembalikan melalui Nginx dan Varnish.

Akses ke sumber data utama ketika data tidak ditemukan di Redis harus mengikuti kode aplikasi yang sebenarnya. Kebijakan cache HTTP dan cache data perlu disesuaikan agar konsistensi data terjaga.

3. Architecture Design

3.1 Diagram arsitektur sistem



Gambar 1. Rancangan arsitektur logis sistem portal berita.

Arsitektur menempatkan Varnish pada lapisan terdepan. Respons yang memenuhi aturan cache dapat dikirimkan langsung ke client, sedangkan permintaan yang memerlukan pemrosesan diteruskan ke Nginx, aplikasi web, dan Redis sesuai kebutuhan.

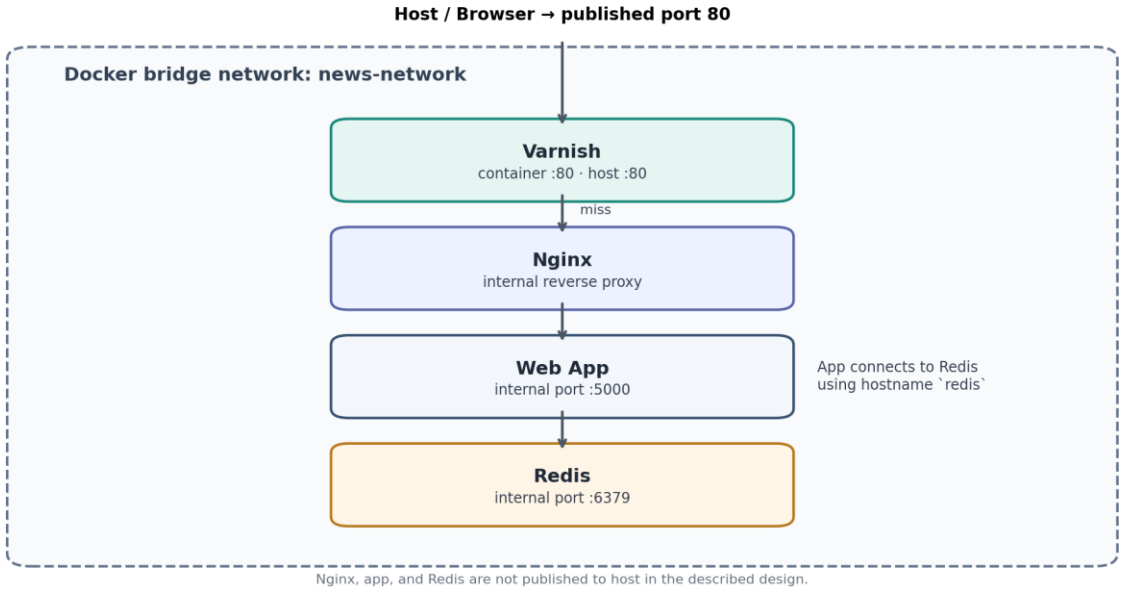
3.2 Peran dan spesifikasi layanan

Layanan	Konfigurasi yang dijelaskan	Peran
Varnish	Image varnish:7.3; host :80 ke container :80; mount ./varnish/default.vcl; cache malloc 128 MB; limit memori 256 MB; CPU 0,50	Edge HTTP cache; bergantung pada health Nginx.
Nginx	Image nginx:alpine; mount ./nginx/nginx.conf read-only; limit	Reverse proxy dan endpoint health check.

	memori 128 MB; CPU 0,50	
Web App	Build ./app; REDIS_HOST=redis; REDIS_PORT=6379; expose :5000; limit memori 512 MB; CPU 1,00	Memproses logika aplikasi dan mengakses Redis.
Redis	Image redis:7-alpine; maxmemory 128 MB; allkeys-lru; limit memori container 192 MB; CPU 0,50	Application data cache dengan kebijakan eviction.

Batas CPU dan memori adalah konfigurasi pembatasan sumber daya, bukan ukuran pemakaian aktual. Perilaku layanan perlu diverifikasi saat container dijalankan.

3.3 Skema jaringan Docker



Gambar 2. Skema konseptual jaringan Docker news-network.

Jaringan menggunakan driver bridge dengan nama news-network. Seluruh layanan terhubung ke jaringan ini dan dapat berkomunikasi menggunakan nama layanan Docker. Port 80 Varnish dipublikasikan ke host, sedangkan port aplikasi dan Redis digunakan untuk komunikasi internal.

3.4 Ringkasan port dan jaringan

Layanan	Port	Akses
Varnish	80	Dipublikasikan sebagai host port 80.
Nginx	Port listener/upstream sesuai nginx.conf	Tidak dipublikasikan langsung dalam rancangan yang dijelaskan.
Web App	5000	Port internal melalui expose.
Redis	6379	Port internal; aplikasi menggunakan hostname redis.

Port listener Nginx perlu diverifikasi terhadap file nginx.conf. Endpoint health check yang disebut dalam konfigurasi adalah http://127.0.0.1:8080/health.

3.5 Dependensi dan health check

Layanan	Dependensi / pemeriksaan
Redis	Health check menggunakan redis-cli ping.
Web App	Bergantung pada Redis; health check melalui endpoint /health

	pada port 5000.
Nginx	Bergantung pada Web App; pemeriksaan melalui endpoint /health yang dikonfigurasi.
Varnish	Bergantung pada status sehat Nginx.

Kondisi service_healthy membantu mengatur urutan startup berdasarkan hasil health check. Namun, mekanisme ini tidak menjamin pemulihan otomatis untuk semua kegagalan setelah startup.

4. Perencanaan Repositori Git

Repositori GitHub menjadi pusat penyimpanan kode, konfigurasi, dan dokumentasi. Pemisahan direktori aplikasi, Nginx, dan Varnish memudahkan anggota memahami tanggung jawab tiap komponen.

Direktori / file	Kegunaan
app/	Kode sumber aplikasi web.
nginx/	Konfigurasi reverse proxy Nginx.
varnish/	Konfigurasi kebijakan cache Varnish.
docker-compose.yml	Definisi layanan, jaringan, dependensi, health check, dan batas sumber daya.
README.md	Deskripsi proyek dan petunjuk penggunaan.
.gitignore	Mengecualikan file yang tidak perlu disimpan di repositori.

4.1 Alur kolaborasi Git

1. Clone repositori ke komputer anggota.
2. Buat branch untuk perubahan atau tugas yang dikerjakan.
3. Lakukan perubahan pada file yang menjadi tanggung jawab anggota.
4. Gunakan commit message yang menjelaskan perubahan.
5. Push branch ke GitHub dan lakukan peninjauan sebelum penggabungan.
6. Sinkronkan perubahan dari repositori bersama secara berkala.

Contoh perintah dasar:

```
git clone https://github.com/febrisariamalia/breaking-news-caching.git
cd breaking-news-caching
git checkout -b docs/architecture
git add README.md
git commit -m "docs: add initial architecture planning"
git push -u origin docs/architecture
```

5. Luaran Pertemuan 1 dan Tahap Berikutnya

Luaran	Bukti dokumentasi
Diagram arsitektur	Gambar 1: client, Varnish, Nginx, aplikasi, dan Redis.
Skema jaringan Docker	Gambar 2: jaringan news-network, koneksi layanan, dan akses port.
Spesifikasi awal Compose	Tabel konfigurasi layanan, dependensi, health check, serta batas sumber daya.
Repositori Git kelompok	URL GitHub, struktur direktori, dan riwayat commit yang relevan.

Tahap berikutnya adalah memvalidasi konfigurasi, menjalankan layanan dengan Docker Compose, memeriksa konektivitas antarkontainer, menguji cache hit dan cache miss, serta menjalankan pengujian beban bertahap. Hasil pengujian perlu dicatat terpisah dari rancangan pada dokumen ini.

Kesimpulan

Pertemuan 1 menghasilkan dasar perencanaan dan rancangan arsitektur Multi-Tier Edge Caching System untuk portal berita nasional. Sistem menggabungkan Varnish, Nginx, aplikasi web, dan Redis dalam container Docker pada jaringan news-network. Docker Compose digunakan untuk mendefinisikan konfigurasi layanan, sementara GitHub mendukung pengelolaan kode dan dokumentasi kelompok. Kemampuan menghadapi lonjakan trafik tetap perlu divalidasi melalui pengujian implementasi.

Referensi

Repositori proyek Breaking News Caching. GitHub. https://github.com/febrisariamalia/breaking-news-caching.git